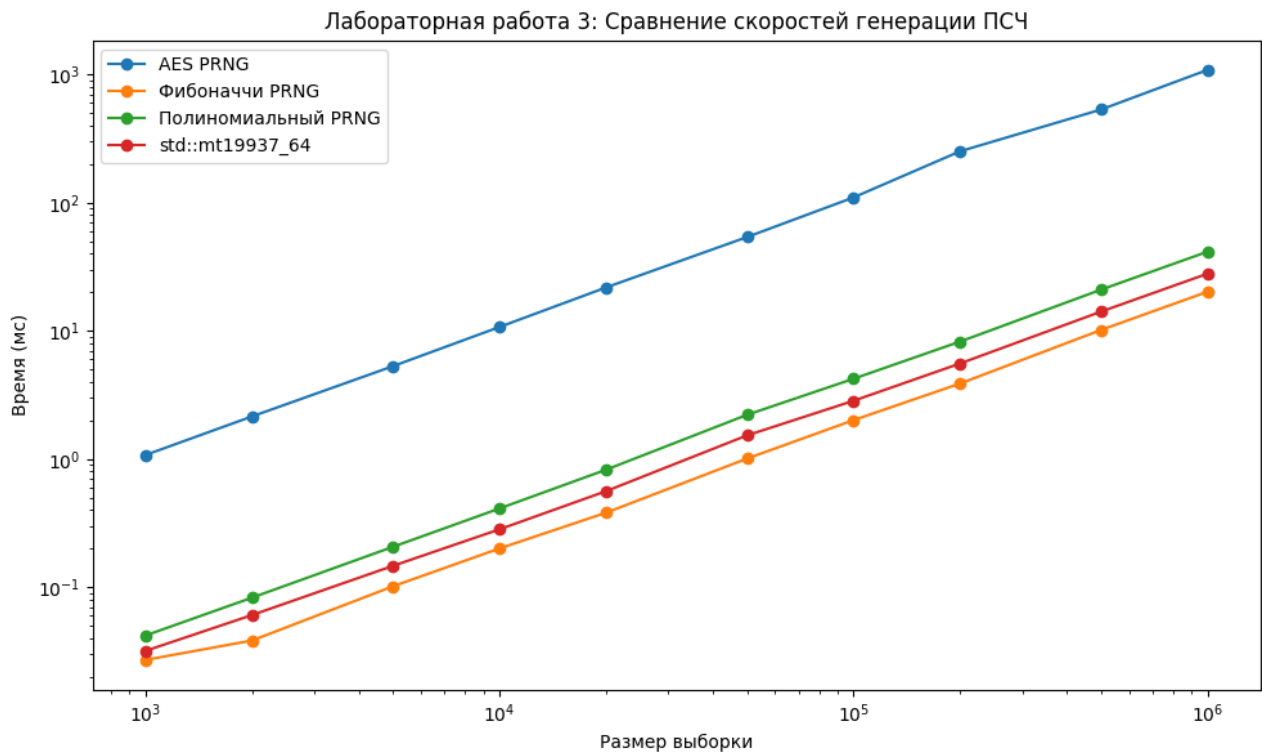


Лабораторная работа 3



Репозиторий: <https://github.com/diduk001/programming-techniques/tree/main/practice3>

Лабораторная работа 3: Генератор псевдослучайных чисел

Общее задание:

- 1) Модифицировать (предложить собственные) три метода генерации псевдослучайных чисел.
- 2) Получить не менее 20 выборок каждым методом (диапазон чисел в каждой выборке не менее 5000) объемом не менее 1000 элементов каждая.
- 3) Для каждой выборки посчитать среднее, отклонение и коэффициент вариации.
- 4) Каждую выборку проверить на равномерность распределения и случайность, используя критерий Хи-квадрат. Сделать выводы о равномерности и случайности генератора.
- 5) Для каждого алгоритма осуществить проверку с помощью не менее 5-ти тестов NIST и/или Diehard. Ссылки на реализацию тестов можно найти тут. Сделать выводы и сравнить их с п.4.
- 6) Засечь время генерации чисел от тысячи до миллиона элементов обоими предложенными методами и любым стандартным методом используемого языка программирования. Построить графики сравнения скоростей в зависимости от объема выборки.

Выбранные алгоритмы:

1) Предложенный метод генерации с использованием элементов AES:

1. Задан параметр Key - матрица байт 4x4 с заранее случайно сгенерированными числами, S - таблица замены, где для каждого байта определено значение, на которое он заменяется
2. Изначальное состояние (seed, 16 байт) записывается в матрицу 4x4. Назовем матрицу состояния State
3. 10 раз повторяется следующее:
 1. Операция SubBytes заменяет каждый байт из State на соответствующий байт согласно S
 2. ShiftRows циклически сдвигает влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3
 3. AddKey поэлементно прибавляет Key к State используя XOR
 4. Состояние возвращается как псевдослучайное число

2) Предложенный метод генерации на основе чисел Фибоначчи

1. Задан параметр A - inicialный параметр генератора
2. Изначальный seed - число, кладется в переменную B
3. Состояние вычисляется согласно модифицированной формулы чисел Фибоначчи: $A, B = B, (A * A) + ((B * B) \ll 17 \mid (B * B) \gg 47)$
4. B возвращается как псевдослучайное число

3) Предложенный метод генерации на основании полиномиального хэширования

1. Задается seed - 8 байт
2. seed побайтово складывается с заранее заданными случайными 8 байтами, чтобы получить состояние State
3. Псевдослучайное число X возвращается как $(State[0] + State[1] * P + State[2] * P * P + \dots + State[7] * (P^7))$
4. State меняется как $State[1] = State[0], State[2] = State[1], \dots, State[7] = State[6], State[0] = X \% 256$

Выводы

Статистические тесты:

AES PRNG Sample Statistics:

Mean: 9053930279102768128.000000
Standard Deviation: 5419451002270816256.000000
Coefficient of Variation: 0.598574
Chi-Square Uniformity Test: 2297.066000
Chi-Square Randomness Test: 0

p-value for NIST tests:

Frequency Test: 0.000000
Block Frequency Test: 0.000000
Runs Test: 0.000000
Longest Run Test: 0.000000

Fibonacci PRNG Sample Statistics:

Mean: 9249605030853058560.000000
Standard Deviation: 5355264365073776640.000000
Coefficient of Variation: 0.578972
Chi-Square Uniformity Test: 19.576000
Chi-Square Randomness Test: 1

p-value for NIST tests:

Frequency Test: 0.545461
Block Frequency Test: 0.504522
Runs Test: 0.972948
Longest Run Test: 0.016950

Rolling PRNG Sample Statistics:

Mean: 9441715994641999872.000000
Standard Deviation: 5411431557224493056.000000
Coefficient of Variation: 0.573141
Chi-Square Uniformity Test: 1075.498000
Chi-Square Randomness Test: 0

p-value for NIST tests:

Frequency Test: 0.000000
Block Frequency Test: 0.999971
Runs Test: 0.000000
Longest Run Test: 0.000000

Лабораторная работа №3

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 AES_PRNG Struct Reference	5
3.1.1 Constructor & Destructor Documentation	5
3.1.1.1 AES_PRNG()	5
3.1.2 Member Function Documentation	6
3.1.2.1 generate()	6
3.2 FibonacciPRNG Struct Reference	6
3.2.1 Member Function Documentation	6
3.2.1.1 next()	6
3.3 RollingPRNG Struct Reference	7
3.3.1 Member Function Documentation	7
3.3.1.1 next()	7
4 File Documentation	9
4.1 src/aes_prng.h File Reference	9
4.1.1 Detailed Description	9
4.1.2 Function Documentation	10
4.1.2.1 AddKey()	10
4.1.2.2 ShiftRows()	10
4.1.2.3 SubBytes()	10
4.2 aes_prng.h	10
4.3 src/fib_prng.h File Reference	11
4.3.1 Detailed Description	11
4.4 fib_prng.h	12
4.5 src/rolling_prng.h File Reference	12
4.5.1 Detailed Description	12
4.6 rolling_prng.h	12
Index	13

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AES_PRNG		5
FibonacciPRNG		6
RollingPRNG		7

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/aes_prng.h	Объявление методов генерации псевдослучайных чисел с использованием алгоритма AES	9
src/fib_prng.h	Объявление методов генерации псевдослучайных чисел с использованием чисел Фибоначчи	11
src/rolling_prng.h	Объявление методов генерации псевдослучайных чисел с использованием полиномиального хеширования	12

Chapter 3

Class Documentation

3.1 AES_PRNG Struct Reference

Public Member Functions

- [AES_PRNG](#) (const uint8_t seed[16])
Инициализировать генератор с помощью начального значения (seed).
- void [generate](#) (uint8_t output[16])
Сгенерировать псевдослучайное число, выполняя 10 раундов алгоритма AES.

Public Attributes

- uint8_t state [4][4]
Матрица состояния 4x4.

3.1.1 Constructor & Destructor Documentation

3.1.1.1 AES_PRNG()

```
AES_PRNG::AES_PRNG (  
    const uint8_t seed[16])
```

Инициализировать генератор с помощью начального значения (seed).

Parameters

seed	Начальное значение (16 байт)
------	------------------------------

3.1.2 Member Function Documentation

3.1.2.1 generate()

```
void AES_PRNG::generate (
    uint8_t output[16])
```

Сгенерировать псевдослучайное число, выполняя 10 раундов алгоритма AES.

Returns

Сгенерированное псевдослучайное число (16 байт)

The documentation for this struct was generated from the following files:

- [src/aes_prng.h](#)
- [src/aes_prng.cpp](#)

3.2 FibonacciPRNG Struct Reference

Public Member Functions

- FibonacciPRNG (uint64_t seed)
Конструктор для инициализации генератора с помощью начального значения (seed).
- uint64_t [next](#) ()
Генерация псевдослучайного числа

Public Attributes

- uint64_t A
- uint64_t B
Изначальный seed, который будет изменяться при генерации псевдослучайных чисел

3.2.1 Member Function Documentation

3.2.1.1 next()

```
uint64_t FibonacciPRNG::next ()
```

Генерация псевдослучайного числа

Returns

Сгенерированное псевдослучайное число

The documentation for this struct was generated from the following files:

- [src/fib_prng.h](#)
- [src/fib_prng.cpp](#)

3.3 RollingPRNG Struct Reference

Public Member Functions

- RollingPRNG (const uint8_t seed[8])
Конструктор для инициализации генератора с помощью начального значения (seed).
- uint64_t [next](#) ()
Генерация псевдослучайного числа

Public Attributes

- uint8_t state [8] = {0xAB, 0xCD, 0xEF, 0x12, 0x34, 0x56, 0x78, 0x90}
Состояние генератора, которое будет изменяться при генерации псевдослучайных чисел

Static Public Attributes

- static const uint8_t P = 37
Произвольное простое число, используемое для полиномиального хэширования

3.3.1 Member Function Documentation

3.3.1.1 next()

```
uint64_t RollingPRNG::next ()
```

Генерация псевдослучайного числа

Returns

Сгенерированное псевдослучайное число

The documentation for this struct was generated from the following files:

- [src/rolling_prng.h](#)
- [src/rolling_prng.cpp](#)

Chapter 4

File Documentation

4.1 src/aes_prng.h File Reference

Объявление методов генерации псевдослучайных чисел с использованием алгоритма AES.

```
#include <stdint>
```

Classes

- struct [AES_PRNG](#)

Functions

- void [SubBytes](#) (uint8_t state[4][4])
Заменить каждый байт из State на соответствующий байт согласно S.
- void [ShiftRows](#) (uint8_t state[4][4])
Циклически сдвинуть влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3.
- void [AddKey](#) (uint8_t state[4][4])
Позлементно прибавить Key к State используя XOR.

4.1.1 Detailed Description

Объявление методов генерации псевдослучайных чисел с использованием алгоритма AES.

1. Задан параметр Key - матрица байт 4x4 с заранее случайно сгенерированными числами, S - таблица замены, где для каждого байта определено значение, на которое он заменяется
2. Изначальное состояние (seed, 16 байт) записывается в матрицу 4x4. Назовем матрицу состояния State
3. 10 раз повторяется следующее:
 - (a) Операция SubBytes заменяет каждый байт из State на соответствующий байт согласно S
 - (b) ShiftRows циклически сдвигает влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3
 - (c) AddKey поэлементно прибавляет Key к State используя XOR
4. Состояние возвращается как псевдослучайное число

4.1.2 Function Documentation

4.1.2.1 AddKey()

```
void AddKey (
    uint8_t state[4][4])
```

Поэлементно прибавить Key к State используя XOR.

Parameters

state	Входная матрица состояния 4x4
-------	-------------------------------

4.1.2.2 ShiftRows()

```
void ShiftRows (
    uint8_t state[4][4])
```

Циклически сдвинуть влево вторую строку матрицы на 1, третью - на 2 и четвертую - на 3.

Parameters

state	Входная матрица состояния 4x4
-------	-------------------------------

4.1.2.3 SubBytes()

```
void SubBytes (
    uint8_t state[4][4])
```

Заменить каждый байт из State на соответствующий байт согласно S.

Parameters

state	Входная матрица состояния 4x4
-------	-------------------------------

4.2 aes_prng.h

[Go to the documentation of this file.](#)

```
00001
00016 #pragma once
00017
00018 #include <stdint>
00019
00020 static const uint8_t key[4][4] = {
00021     {0x72, 0xdd, 0xcb, 0x1f},
00022     {0x7a, 0xb4, 0x7b, 0xa0},
00023     {0x6f, 0x32, 0x48, 0xef},
00024     {0xa6, 0xad, 0x8e, 0xe4}};
00025
```

```

00026 static const uint8_t sbox[256] = {
00027     // 0   1   2   3   4   5   6   7   8   9   A   B   C   D   E   F
00028     0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
00029     0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,
00030     0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,
00031     0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
00032     0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,
00033     0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
00034     0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,
00035     0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,
00036     0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,
00037     0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,
00038     0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,
00039     0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,
00040     0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
00041     0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,
00042     0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,
00043     0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16};
00044
00049 void SubBytes(uint8_t state[4][4]);
00050
00055 void ShiftRows(uint8_t state[4][4]);
00056
00061 void AddKey(uint8_t state[4][4]);
00062
00063 struct AES_PRNG
00064 {
00065     uint8_t state[4][4];
00066
00071     AES_PRNG(const uint8_t seed[16]);
00072
00077     void generate(uint8_t output[16]);
00078 };

```

4.3 src/fib_prng.h File Reference

Объявление методов генерации псевдослучайных чисел с использованием чисел Фибоначчи

```
#include <stdint>
```

Classes

- struct [FibonacciPRNG](#)

Variables

- const uint64_t A_initial = 0x123456789abcdef0
Изначальный параметр генератора

4.3.1 Detailed Description

Объявление методов генерации псевдослучайных чисел с использованием чисел Фибоначчи

1. Задан параметр A - изначальный параметр генератора
2. Изначальный seed - число, кладется в переменную B
3. Состояние вычисляется согласно модифицированной формулы чисел Фибоначчи: $A, B = B, A * A \wedge B * B$
4. B возвращается как псевдослучайное число

4.4 fib_prng.h

[Go to the documentation of this file.](#)

```
00001
00010 #pragma once
00011
00012 #include <stdint>
00013
00014 const uint64_t A_initial = 0x123456789abcdef0;
00015
00016 struct FibonacciPRNG
00017 {
00018     uint64_t A, B;
00019
00020     FibonacciPRNG(uint64_t seed) : A(A_initial), B(seed) {}
00021
00022     uint64_t next();
00023 };
```

4.5 src/rolling_prng.h File Reference

Объявление методов генерации псевдослучайных чисел с использованием полиномиального хеширования

```
#include <stdint>
```

Classes

- struct [RollingPRNG](#)

4.5.1 Detailed Description

Объявление методов генерации псевдослучайных чисел с использованием полиномиального хеширования

1. Задается seed - 8 байт
2. seed побайтово складывается с заранее заданными случайными 8 байтами, чтобы получить состояние State
3. Псевдослучайное число X возвращается как $(\text{State}[0] + \text{State}[1] * P + \text{State}[2] * P * P + \dots + \text{State}[7] * (P^7))$
4. State меняется как $\text{State}[1] = \text{State}[0]$, $\text{State}[2] = \text{State}[1]$, ..., $\text{State}[7] = \text{State}[6]$, $\text{State}[0] = X \% 256$

4.6 rolling_prng.h

[Go to the documentation of this file.](#)

```
00001
00013 #pragma once
00014
00015 #include <stdint>
00016
00017 struct RollingPRNG
00018 {
00019     const static uint8_t P = 37;
00020     uint8_t state[8] = {0xAB, 0xCD, 0xEF, 0x12, 0x34, 0x56, 0x78, 0x90};
00021
00022     RollingPRNG(const uint8_t seed[8]);
00023
00024     uint64_t next();
00025 };
```

Index

- AddKey
 - aes_prng.h, [10](#)
- AES_PRNG, [5](#)
 - AES_PRNG, [5](#)
 - generate, [6](#)
- aes_prng.h
 - AddKey, [10](#)
 - ShiftRows, [10](#)
 - SubBytes, [10](#)
- FibonacciPRNG, [6](#)
 - next, [6](#)
- generate
 - AES_PRNG, [6](#)
- next
 - FibonacciPRNG, [6](#)
 - RollingPRNG, [7](#)
- RollingPRNG, [7](#)
 - next, [7](#)
- ShiftRows
 - aes_prng.h, [10](#)
- src/aes_prng.h, [9](#), [10](#)
- src/fib_prng.h, [11](#), [12](#)
- src/rolling_prng.h, [12](#)
- SubBytes
 - aes_prng.h, [10](#)